Route Optimization for Emergency Vehicles Analyzing Spatial and Temporal Dependencies using Graph Convolutional Network

Faria Sultana[1], Sadia Mubasshira Rahman[1], M. A. H. Akhand[1], Sk Imran Hossain[1] and Md Abdus Samad Kamal[2]

[1]Department of Computer Science and Engineering, Khulna University of Engineering & Technology, Khulna -9203, Bangladesh, [2]Graduate School of Science and Technology, Gunma University, Kiryu 376-8515, Japan

Abstract

Ensuring quick emergency vehicle response is essential for efficient service in crowded urban locations. Identification of optimal route for an emergency vehicle in the busy area is always a critical issue due to heterogeneous traffic on road. Traditional route optimization techniques prioritize the most cost-efficient routes, frequently ignoring the time factor which is vital in emergency situations. The aim of this study is to address the challenge through an emergency vehicle route optimization (EVRO) system that focuses on minimizing travel time based on road conditions. The system leverages the A* search algorithm to creatively modify its edge weights in order to determine the most optimal route. Graph Convolutional Network (GCN) and GraphSAGE are utilized in predicting the travel times based on road conditions, and these predictions are used as weights in the A* algorithm. The efficacy of the system is demonstrated by evaluations conducted on the German HighD dataset and in Khulna City, Bangladesh. Comparative studies with standard A* reveal different routes during peak hours, and the suggested approaches consistently outperform conventional A*, with GraphSAGE+A* producing the best results. This inventive EVRO method conveys its potential to improve the effectiveness of emergency response in city environment.

1 Introduction

Emergency vehicles, such as ambulances, fire trucks, and police units, play a crucial role in responding to critical situations, including medical emergencies, accidents, fires, and natural disasters. However, navigating through congested or crisis-affected areas poses significant challenges. Factors such as overcrowding, unplanned infrastructure, and traffic congestion can hinder timely arrivals,

potentially exacerbating the severity of crises. In fire emergencies, the urgency is even greater, as fires can double in size every 17 seconds[1]. Unplanned infrastructure, particularly in densely populated urban areas and remote regions, further complicates emergency responses by restricting access to necessary resources.

To address these challenges, several strategies have been proposed, including advanced traffic management systems, dedicated emergency lanes, public awareness campaigns, improved infrastructure, GPS-based route optimization, and collaboration between authorities. Additionally, legislative support can enhance the efficiency of emergency vehicle navigation, ultimately improving response times and reducing damage. For example, the study in [2] utilized GIS-based network analysis to optimize fire service routes in a densely populated urban area, resulting in improved response times and service coverage.

In emergencies, rapid response times are critical to preventing further deterioration of victims' conditions. For instance, in cardiac arrest cases, the first hour—often referred to as the "golden hour"—is crucial. The survival rate decreases by 10-12% for every minute without electrical defibrillation, but early administration of basic life support (BLS) can reduce this decline to 3-4% [3] Similarly, in fire emergencies, swift intervention is essential to prevent small fires from escalating uncontrollably [1].

Route optimization is key to ensuring that emergency vehicles reach their destinations in the shortest possible time. One of the biggest challenges in this process is accurately predicting road conditions in real time. Emergency vehicles often face unexpected delays due to rapidly changing traffic situations, roadblocks, or congestion, leading to increased response times and fuel consumption. Additionally, ensuring minimal disruption to regular traffic while providing a clear path for emergency vehicles is difficult, particularly in regions where pre-clearing roads is impractical. The primary objective in such scenarios is to determine optimal routes by rapidly predicting travel times and road conditions.

Previous research on emergency vehicle route optimization has primarily focused on predicting travel times and congestion levels [4], optimizing rescue operations at accident sites [5], and pre-clearing lanes for emergency vehicles to travel unimpeded [6] Data-driven approaches have also been explored for selecting optimal ambulance routes while using blue lights and sirens [7]. Another study introduced a short-term traffic flow prediction system to assess road conditions and determine the fastest routes [8].

While pre-clearing roads for emergency vehicles has proven effective in some cases [1]this approach is less feasible in densely populated and poorly planned cities like those in developing countries, such as Bangladesh. In such settings, pre-clearing roads could lead to traffic system collapse [9]. Instead, a more practical approach involves analyzing road conditions in advance to determine the optimal route. Research has demonstrated that traffic data is highly dependent on both spatial and temporal factors [10], with road conditions varying based on time and location. Therefore, an effective routing system must consider these dependencies. Graph Convolutional Networks (GCNs) have emerged as

a promising solution for analyzing spatial and temporal dependencies in traffic data. These neural network architectures can model complex traffic patterns and improve route optimization by leveraging both geographical and temporal information.

This study aims to develop an optimal route selection model for emergency vehicles, ensuring minimal travel time while avoiding congested or blocked roads. The key objectives include:i) Identifying the optimal route for emergency vehicles.ii) Avoiding heavily congested or unavailable roads. iii) Minimizing detours to ensure faster arrival times. iv) Optimizing routes without significantly disrupting normal traffic flow.

This study employs a deep learning-based approach using GCN to enhance the accuracy of travel time predictions, making it a valuable tool for emergency vehicle routing. A graph-based model has been developed to identify the most efficient emergency routes by considering various traffic and road conditions. The road network dataset has been transformed into a graph representation, and the effectiveness of the proposed routing system has been evaluated based on travel time. The results demonstrate that the suggested approach significantly improves emergency response times.

The remainder of this paper is organized as follows: Section 2 discusses background studies and literature review on emergency vehicle route optimization. Section 3 presents the proposed methodology, detailing the system overview and the techniques used. Section 4 outlines the experimental setup and results, followed by an analysis of the findings. Finally, Section 5 concludes the paper, summarizing key insights and suggesting directions for future research.

2 Literature Review / Background Studies and Related Studies

Emergency vehicle route optimization has been widely studied due to its critical role in saving lives during crises. Many studies have focused on specific situations or regions, yet common trends can be observed. The general approach involves analyzing traffic data to identify key influencing parameters, making accurate predictions, and selecting the most efficient routes accordingly.

Several parameters influence emergency vehicle routing, and identifying the most significant ones is a key challenge. Factors such as the number of connecting roads, real-time vehicle density on specific streets, and spatio-temporal dependencies play crucial roles in optimizing emergency response routes [11]. Traffic data analysis considers three main aspects: time, space, and congestion attributes [8] The paths between the source and destination are the most critical components, with road capacity, speed limits, and traffic density directly affecting travel times. The density and size of vehicles on roads impact overall congestion levels. Moreover, traffic patterns fluctuate based on the time of day and the day of the week, significantly affecting emergency response efficiency in urban environments [12] For emergency material distribution, four factors such

as distribution time window, vehicle loading, dynamic real-time demand, and distance traveled are considered [13].

Table 1: Summary of some Machine Learning based approaches

Work Reference, Year	Dataset Used	Methodology	Contribution
[4], 2017	Expressway network of Beijing	Kernel clustering algorithm, shuffled frog leaping algorithm	Considers the dynamic time-varying characteristics
[14], 2014	US road networks obtained from DIMACS	Dijkstra's algorithm and A* search algorithm with time window	Generalized ALT algorithm and A* search algorithm for the static problem
[15], 2012	Google Earth and Bakırköy Municipality, Istanbul	Improved Genetic Algorithm	Minimized travel time and maximized number of causalities in ambulance
[7], 2019	The London Ambulance System dataset	Nelder-Mead Algorithm, HMM/V map matching, Dijkstra's algorithm	First attempt to data-driven methodology for route selection
[16], 2019	Real-world dataset from China	Genetic Algorithm	Distribution of relief materials in a specified time window
[17], 2016	Simulated dataset of Brazil	Improved Genetic Algorithm	Minimized the total cost

2.1 Machine Learning Based Approaches

Many researchers have leveraged machine learning techniques for emergency vehicle routing. Zhao et al.[4] developed a two-step optimization algorithm integrating the shuffled frog-leaping algorithm and the K-paths algorithm. Poulton et al.[7] employed a predictive algorithm to identify routes with high similarity scores and reduced travel times. Yuan et al. [16] utilized a genetic algorithm to optimize emergency vehicle routing by minimizing response times to distribution points. Similarly, Qin et al. [17] applied a genetic algorithm to enhance response efficiency in disaster scenarios. Branchini et al. [18]introduced a granular local search heuristic that adjusts search space size based on time constraints. Zhang et al. [19] implemented ant colony optimization to reduce relief distribution time in disaster-affected areas. Kula et al. [15] optimized ambulance dispatch by maximizing the number of casualties transported per trip using genetic algorithms. Gharib et al [20]. formulated a heterogeneous routing problem and solved it using the Non-dominated Sorting Genetic Algorithm II (NSGA-II) and the Multi-Objective Firefly Algorithm (MOFA). Liu et al. [21] proposed an Density-Based Spatial Clustering of Application with Noise for passenger information clustering in order to identify the more influencing factors. A summary

of these machine learning-based approaches is presented in Table 1.

Table 2: Summary of some Deep Learning based approaches

Work Reference, Year	Dataset Used	Methodology	Contribution
[5], 2019	Hypothetical emergency event on Shantou-Kumming expressway, Anhui Province	Temporal Convolutional Network, Dijkstra algorithm	Minimized time cost and improved rescue process
[22], 2019	Lankershim Data dataset	Deep Belief Network and LSTM	Improved the accuracy of vehicle position prediction
[23], 2020	Multisource traffic data provided by Shaanxi traffic management bureau, China	Improved Deep Belief Network	Proposed a non-video-based, cost-effective method for identifying traffic flow
[24], 2022	4 Datasets: PEMS03, PEMS04, PEMS07, PEMS08	Interactive dynamic graph convolution structure	Analyzed the nature of traffic data
[25], 2023	6th Beijing road traffic dataset	LSTM and Deep Generative model	Improved prediction accuracy by modeling road segment relationships

2.2 Deep Learning Based Approaches

Deep learning has also been extensively explored for emergency vehicle routing. Jin et al. [5] applied a Temporal Convolutional Network (TCN) to reduce emergency vehicle travel times to accident sites. Jiujun et al. [22] incorporated vehicle body attributes, road conditions, and environmental factors into their Deep Belief Nets and LSTM-based LPMV model for urban traffic prediction. Aoyu et al. [24] introduced STIDGCN, a Spatial-Temporal Interactive Dynamic Graph Convolutional Network, to effectively capture traffic data dependencies. Redouane et al. [26] proposed a three-stage framework using a cross-correlation LSTM model for road segment prediction, deep generative models (DGMs) for data augmentation, and an interpretable Time Series model (N-BEATS) for overall prediction. Van-Thuan et al. [27] designed an ensemble model, SirenNet, which combines CNN-based WaveNet and MLNet to classify traffic sounds, including sirens and vehicle horns. Ping et al. [28] employed an improved Deep Belief Network (DBN) to reconstruct and predict fine-grained traffic flows, excelling in long-term predictions and heavy vehicle assessments. Zongtao et al. [25] developed a ConvLSTM-based hybrid model to predict city-wide taxi origin-destination (OD) flows while integrating spatial-temporal dependencies and travel time correlations. A summary of these deep learning-

based methods is presented in Table 2.

2.3 Observation from Existing Studies

Despite advancements in emergency vehicle routing, several limitations remain. Some studies do not account for location-specific parameters [4], while others propose lane pre-clearing solutions that are impractical for highly congested urban areas [6].Additionally, fuzzy system-based models face decision-making uncertainties [8]. Nasser et al. [14] introduced a time-based shortest path algorithm, assigning time as a weight in routing decisions. Azlin et al. [23] minimized response times using the A* search algorithm to determine optimal emergency routes. Jian et al. [29] optimized relief distribution locations using enhanced monarch butterfly optimization, though achieving global optimality remains challenging. Prakash et al. [30] demonstrated a 48% reduction in average emergency vehicle travel times using a decentralized radio frequency (RF) and GNSS-based geofencing approach. Van-Linh et al. [31] highlighted the effectiveness of proactive route planning and traffic control in reducing emergency response times in congested areas, though careful traffic regulation is necessary to prevent excessive delays for other vehicles. Gaby et al. [32] proposed an Integer Linear Programming (ILP) model to optimize intra-link paths, reducing confusion for non-emergency vehicles. However, sensitivity analysis revealed challenges on smaller roads with high non-ERV densities, along with increased computation times affecting real-time applicability.

2.4 Broader Emergency Management Challenges and Techniques

Emergency management during crises extends beyond route optimization, requiring coordination among stakeholders to address logistical and regulatory challenges. Effective collaboration mitigates spatial mismatches and resource shortages, as highlighted by a study on cross-regional emergency supply dispatch [33], which emphasized the role of higher-level administrative regulation (HAE) in improving resource distribution through supervision, financial incentives, and penalties. Weiqi et al. [34] reviewed traffic control strategies, including route optimization, signal preemption, and lane reservation, advocating for dynamic real-time solutions to minimize emergency vehicle response times. Ryo et al. [35] introduced deterministic algorithms leveraging real-time vehicular ad-hoc network (VANET) data, reducing arrival times by 10% compared to Geocast. Omar et al. [36] proposed an IoT-enabled emergency service system with an NR-index algorithm and VANETs to optimize ambulance routes by balancing distance and congestion. Pengying et al.[37] analyzed geometric and traffic flow factors affecting lane-changing rates at weaving segments, showing that segment length, width, and traffic conditions influence lane-changing behaviors and road safety.

3 Emergency Vehicles using GCN/GraphSAGE and A* Search Algorithm

The timely arrival of emergency vehicles is crucial as delays can result in severe consequences. Therefore, there is a major need to optimize the route of emergency vehicles. This study aims to predict the time needed for emergency vehicles to pass through each track on a road network and identify the quickest route to reach their destination. We have proposed a solution by predicting the travel time with GCN and GraphSAGE and searching the route with shortest travel time using A* search algorithm.

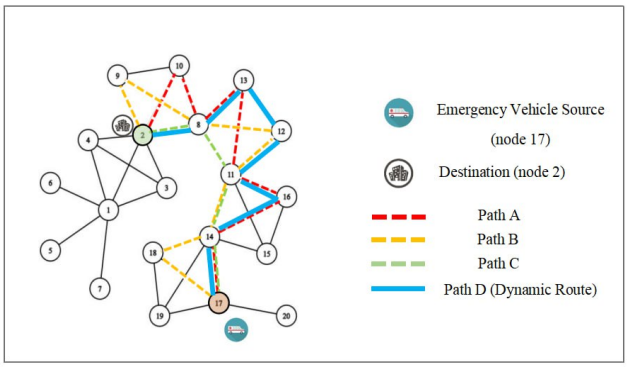


Figure 1: An overview of the proposed system

The proposed model is designed for the dynamic routing of emergency vehicles. At first, a graph network is constructed where each node in the graph is an intersection of four road tracks and the edges of the graph are the tracks from one intersection to another. By utilizing the A* search algorithm with predicted time as the weights, it generates optimal routes dynamically. The system adapts to changing road conditions, suggesting alternative routes when necessary if dynamic traffic conditions are fed to the model. Graph Convolutional Network or GraphSAGE models are employed to predict the time needed for each route. The workflow diagram (Fig. 2) provides a comprehensive understanding of the system's operations.

3.1 Road Network Data Processing

The work involves generating a graph from the road network of six different locations and implementing Graph Convolutional Network and GraphSAGE models separately to predict the time needed for passing each track. The features of the dataset that we've taken into considerations are listed below:
Driving direction: The driving direction of the vehicle. Either 1 for the left direction (upper lanes) or 2 for the right direction (lower lanes).
The "tracks" files of the dataset have a column named "xAccelaration" which

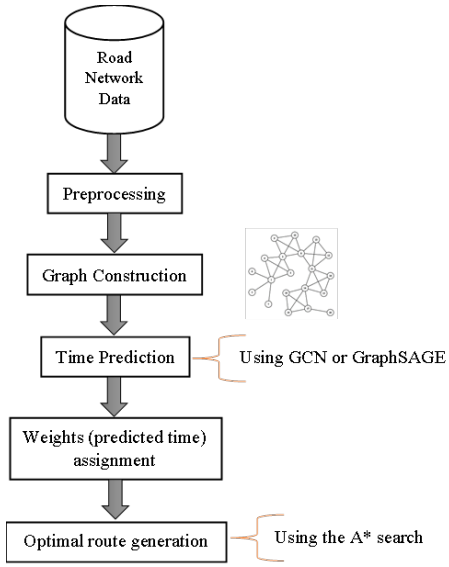


Figure 2: Workflow diagram of the proposed system.

shows the accelaration of each car along the horizontal axis. The accelaration along the vertical axis represented by "yAccelaration" was considered negligible. We calculated the mean accelaration from the "xAccelaration" for each car. The "tracksmeta" files has the "meanXVelocity" column, we used velocity from this column for each car. The length of each track is 420. Equation 3.1 is applied to calculate the time.
Traveled distance (c) : The distance covered by the track.
Mean velocity (b) : The mean velocity in driving direction.
Mean acceleration (a) : The mean acceleration in driving direction.
Time (x) was calculated from these features utilizing this (3.1) equation:

$$X = \frac{b - +\sqrt{b^2 - 4ac}}{2a}$$

The predicted time from the models was compared with the calculated time to measure the performance of both models. The A* search algorithm were then used to generate the quickest path based on the predicted time. Standardization: Standardization, commonly referred to as Z-score normalization, is a preprocessing step used in many data analysis techniques to scale a feature to a standard scale, for instance, the standard normal distribution with a mean of zero and a standard deviation of one. In order to produce more precise and reliable results, it is assured that no aspect of the data set has more influence than any other. The standardization formula is written as:

$$Z = (X - \mu) / \sigma$$

Where Z is the standardized feature, X is the original feature, μ is the feature's mean, and σ is the standard deviation of the feature. The data was normalized by standardizing it. This process was done to ensure that every feature has a similar range, which enhances the model's accuracy. **Inference for Missing Values:** The mean values of each column were used to fill in any missing values that were present in the data. This process ensured that the model contained complete data for all samples.

3.2 Road Network to Graph Construction

The traffic data from several locations contained a road network data unique to each location. For each location, we transformed the data into an individual graph network. Therefore, the graph of each location was represented by an edge list where the edges between all intersection can be found. An edge is a track which is represented by (a,b) where 'a' and 'b' are nodes(intersection of tracks). Generation of input graph: The algorithm generates a set of edges between nodes in a graph for each different location of the dataset. It constructs a graph by connecting nodes in a recursive manner. The number of nodes depends on the data given for each location. The process can be described as follows:

Algorithm 1: Graph Construction Algorithm

```
Input: Number of edges E
Output: Edges of the constructed graph
Initialize empty list edges ;
Initialize count list of zeros to track edge count per node ;
Calculate number of nodes N such that each node has 4 edges;
for each node i from 1 to N do
    Select three unvisited nodes ;
    for each selected node j do
        if node j has fewer than 4 connected edges then
            Connect node i and node j ;
            Append edge (i, j) and (j, i) to edges ;
            Increment edge counts for i and j in count list ;
Store all the edges;
```

3.3 Time Prediction from Constructed Graph

Graph based Neural Networks tend to perform better in road network as road networks itself are a graph. This ensures a robust and comprehensive analysis of complex spatial relationships within the road system. Base on this, both GCN and GraphSAGE models were chose separately to train the dataset to predict the time required to pass each track in this system.

3.3.1 Time Prediction with GCN

The Graph Convolutional Network (GCN) leverages graph structures to perform convolutional operations. The model consisted of multiple GCNConv layers, interleaved with activation functions (ReLU), dropout regularization and a final linear layer. The architecture of the proposed GCN model is as follows:
The proposed architecture consists of four GCNConv layers and two linear layers. In the first GCNConv layer, the input node feature tensor and edge index tensor are passed through the layer to transform it into a tensor with 100 dimensions. This transformed tensor is then passed through the second GCNConv layer, reducing the dimension space to 16. The third and fourth GCNConv layers follow the same pattern as the second GCNConv layer and further reduce the feature space to 8 and 4, respectively. Then, a linear layer is applied to the output of the fourth GCNConv layer, reducing the feature space to 4. Finally, the original node features are passed through another linear layer, transforming them into the final prediction of the time required to pass each track. The model uses the output of the second linear layer as the final prediction. The activation function used is ReLU, which is applied after the first and second GCNConv layers. Dropout regularization with a value of 0.5 is applied after the first and second GCNConv layers to prevent overfitting. Architecture of the proposed GCN is depicted in Fig. 3.

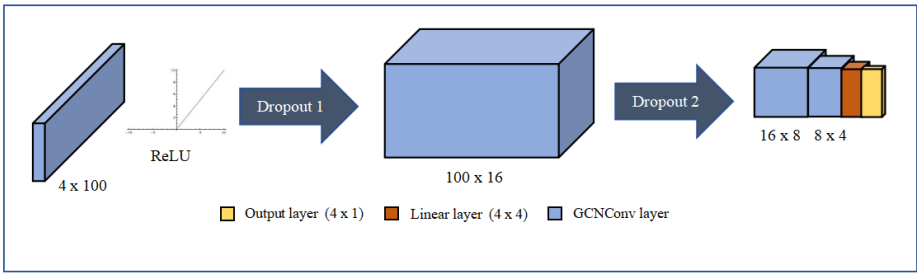


Figure 3: Architecture of the proposed GCN model.

The GCN model takes in a graph with features and edge information. The input features are passed through four GCNConv layers with ReLU activation and dropout regularization. The output of the last GCNConv layer is passed through a linear layer with four output features and then to another linear layer with a single output. This final output represents the predicted travel time between the source and destination tracks.
During each training epoch, the GCN model takes the input data, which includes the node features and edge information, and performs forward propagation through its layers to generate a predicted output. The predicted output is then compared to the actual target values to calculate the loss. The error is then back-propagated through the layers of the GCN model to compute the gradients of the loss with respect to the model parameters. Stochastic Gradient

Descent is used to update the model parameters in the direction of the computed gradients. The model is trained by minimizing the Mean Absolute Error and the Mean Squared Error loss between the predicted and calculated travel time. This process is repeated for 1000 epochs.

3.3.2 Time Prediction with GraphSAGE

GraphSAGE stands for Graph Sample and Aggregated is a type of Graph Neural Network (GNN). It is designed for large-scale graphs or network systems. It can be aggregated with various methods such as mean, LSTM-style, pooling etc which makes it dissimilar to GCN. It considers mini-batch samples and aggregated information instead of considering the whole graph at once. The proposed GraphSAGE model consists of a series of SAGEConv layers and a linear layer for prediction. Each graph convolutional layer uses the SAGEConv class to perform graph-based convolution on the input node features and graph structure. The first convolutional layer has an input channel equal to the number of node features in the graph and 100 output channels. The activation function ReLU is applied after each convolutional layer to introduce non-linearity in the model. A dropout layer is also added after each convolutional layer to prevent overfitting. The forward method takes in graph data and performs the forward computation by passing the node features through the defined layers. The final linear layer transforms the output of the last convolutional layer into a single value representing the predicted time required to pass each track.
This model uses graph convolutions to extract features from the graph structure and node features then outputs a prediction of the time required to pass each track. After predicting the amount of time required to pass each track, an average of all the estimated values is calculated. This timing is regarded as the most accurate projection for each track. As a result, each track achieves a single time value. The proposed GraphSAGE architecture is depicted in Fig. 4.

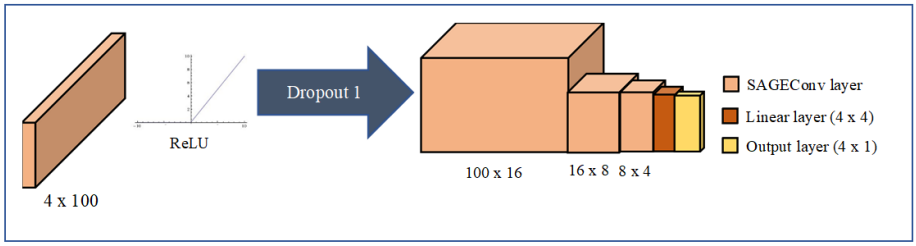


Figure 4: Architecture of the proposed GraphSAGE model.

The model is performing graph convolutional operations using the GraphSAGEConv layers to perform node-level classification. The model takes as input the node features and the edges representing the graph structure. The input

tensor is passed through a series of four GraphSAGEConv layers, where each layer transforms the input feature tensor into a smaller feature space. The output of the fourth GraphSAGEConv layer is then passed through a linear layer to further reduce the feature space to 4 features. Finally, another linear layer is applied to transform the 4-dimensional feature space into the final prediction of the time required to pass each track. During training, the model computes the predicted output by forward propagation. The loss between the predicted output and the actual target values is calculated and backpropagation is performed to compute the gradients of the loss with respect to the model parameters. Stochastic Gradient Descent is then used to update the model parameters in the direction of the computed gradients. This process is repeated for 1000 epochs.

3.4 Predicted Weight Assignment

In section 3.3, the process of time prediction is described. Upon obtaining the predicted time to cross each road, these values are considered as the weights for the graph. Subsequently, these weights are assigned to the constructed graph derived from the road network (as outlined in section 3.2). The generated edges and vertices are already in place. Following the assignment operation, the graph is prepared for the generation of the optimal route.

3.5 Optimal Route Generation using A* Search Algorithm

To determine the most optimal route based on the shortest time to travel from a given source to a particular destination, the shortest path algorithm, A* search is used. The A* search algorithm is popular for finding the shortest route between two nodes in a graph. The algorithm uses a combination of the breadth-first search algorithm and a heuristic function to find the optimal path. The system stores the nodes of a graph and the time required to travel between each connected pair of nodes as the cost. The time requirement between each pair of nodes, the initial track number (source) and the final track number (destination) are the input parameters of A* search algorithm. In addition to these parameters, the algorithm required a heuristic parameter, which was also passed as input. The heuristic is an additional estimate of the distance between the current node being evaluated and the destination node. It helps the algorithm to focus on exploring the most promising paths toward the destination, instead of exploring all possible paths. Using these parameters, the algorithms generated the optimal path with the minimum travel time from the source to the destination.

3.6 Final Route Selection for Emergency Vehicles (EV) Based on Priority Scoring

In the context of optimizing routes for Emergency Vehicles (EVs), the primary objective is to ensure that these vehicles can traverse from their origin to the destination in the least possible time, considering the unique requirements of emergency situations. The process involves not only selecting the shortest or fastest route but also factoring in conditions that favor rapid and unhindered passage. Given the critical nature of emergency response, certain road segments may be more favorable for EVs based on specific characteristics such as higher average velocity, fewer lane changes, and greater safety metrics. To quantify these characteristics, a priority scoring system is applied to each route. The following steps outline the process of route selection based on prioritization for EVs:

1. Extract Features:

meanXVelocity: The average velocity of vehicles in the segment.
maxXVelocity: The maximum velocity of vehicles in the segment. Segments with higher velocities are preferred as they indicate smoother and faster traffic flow.
numLaneChanges: The number of lane changes made by vehicles in the segment. Fewer lane changes imply a less congested route, reducing the likelihood of delays.
minTTC (Time to Collision): The minimum time to collision, which indicates safety levels. Segments with higher safety scores are prioritized to minimize the risk of accidents or delays.

2. Calculate Priority Score:

priority_score = (mean_x_velocity + max_x_velocity) / 2 - num_lane_changes - (1 / min_ttc)

(meanXVelocity + maxXVelocity) / 2: This gives an average velocity for the segment, favoring segments where vehicles move quickly, which is typically better for EVs.
- numLaneChanges: A negative factor is applied to segments with more lane changes, as frequent lane changes can indicate traffic congestion or road complexity, which is less favorable for EVs.
- (1 / minTTC): A negative factor is applied to segments with low Time to Collision values. A lower TTC is riskier, so segments with higher safety (higher minTTC) are preferred.
The priority score is returned for each segment, which is then used to evaluate and compare different routes. This scoring helps identify which routes are more likely to provide a fast and uninterrupted journey for EVs. This approach uses available data to logically infer conditions that would favor EVs, such as speed, safety, and road smoothness, without requiring additional data. It's simple to implement and aligns with the goal of optimizing routes for emergency

situations.

3.7 Significance of The Proposed Model

The following are the significant points of the proposed system:
1.Graph Construction: By creating separate graphs for each location, the unique features of the road network were captured and better modelled the emergency vehicle response time problem. This approach generated more accurate predictions of the time needed to pass each track by implementing Graph Convolutional Network and GraphSAGE models separately. Thus, constructing a graph for each location provided a solution to the problem of emergency vehicle response time and showcased the potential of graph-based models in transportation engineering.
2.Computational efficiency and flexibility: Using the A* search algorithm and the predicted time as weights provide a computationally efficient way to calculate the quickest path. The proposed model can be easily applied to different road networks and emergency service settings as well.
3.Novelty: The use of graph convolutional network and GraphSAGE models for predicting the time needed to pass each track and incorporating them into the shortest path algorithm is a novel approach that has the potential to outperform traditional methods. In this approach, the shortest path algorithm finds the quickest route having time as inputs, not the shortest distance.
4.Potential impact on transportation and logistics: The proposed system can significantly impact transportation and logistics by providing more accurate and optimized routes, reducing travel time, and improving overall efficiency.
5.Time optimization: The use of A* algorithm to find the optimal path based on time ensures that the proposed system provides the quickest route from a source to a destination, taking into account the time required for passing each track.

4 Experimental Studies

In this section, the experimental results and analysis of the proposed models are presented. First, an overview of the experimental setup is provided, including the dataset, preprocessing steps, and hyperparameters used to train the models. Next, the experimental results, including the performance of the GraphSAGE and GCN models on the evaluation metrics used, are presented. The performance of the models is also compared in terms of their convergence speed and stability. Following this, the results were analyzed and were used to draw insights into the effectiveness of GraphSAGE in comparison to GCN. The results obtained from the experiments are discussed in detail, and their significance is explained. The analysis findings are used to conclude the effectiveness of the proposed models and their potential for future research.

4.1 Experimental Setup

The experimental setup involves training and evaluating two different graph convolutional neural network models - GCN and GraphSAGE - to predict the time needed to pass each track in six locations. Both models have four conv layers with 100, 16, 8, and 4 features respectively. The activation function used is ReLU. The GCN model uses GCNConv layers, while the GraphSAGE model uses SAGEConv layers. The final prediction is made using a linear layer.

Training method: The training was performed using the Stochastic Gradient Descent (SGD) optimizer with a learning rate of 0.01 and a weight decay of 0.00005.

Parameter and Loss function: The models were trained for 2000 epochs. At each epoch, the optimizer's gradients are zeroed. Then, the model is run on the input data, and the output is compared with the true values using both Mean Absolute Error (MAE) loss function and Mean Squared Error (MSE) loss function. The loss is then back-propagated through the model, and the optimizer updates the model's weights with each epoch. The R-squared value is calculated during each epoch to evaluate the model's performance. The R-squared value is calculated by first finding the residual sum of squares (RSS) and the total sum of squares (TSS). The RSS is the sum of squared differences between the predicted output and the true output, while the TSS is the sum of squared differences between the true output and the mean of the true output. The R-squared value is then calculated as one minus the ratio of RSS to TSS.

Model selection: The best-performing model was selected based on the lowest mean absolute error (MAE) and mean squared error (MSE) on the validation set.

4.2 Performance Analysis on Benchmark Data

4.2.1 Benchmark Dataset

We used German highD dataset in this work [38]. The Highway Drone (highD) Dataset is a dataset of naturalistic vehicle trajectories recorded on German highways. It is a large-scale dataset with 110,500 vehicles, 44,500 driven kilometers, and 147 driven hours. Locations of which traffic data are obtained are depicted in Fig 5. The dataset contains the information in three sub-datasets. They are:
Recording Meta Information: Here, the information of the recording is stored. We have used the frame-rate shown here for the time-based properties.

Track Meta Information: This file holds the overview of all tracks. It has values such as the speed of each track or the distance covered.
Tracks: This file contains the time-dependent data of each vehicle on each track. The dataset contains high-quality and a variety of features such as:
1. Six different recording locations

Figure 5: Locations of which traffic data are obtained.

Table 3: Information about the Benchmark Data.

Location id	Total Number of vehicles	Total Driven Distance[m]	Number of tracks considered
1	50760	20521751.01	1410
2	1828	735993.26	914
3	2326	939787.2	1163
4	2565	1023818.05	855
5	5463	2197365.41	607
6	1428	565742.51	714

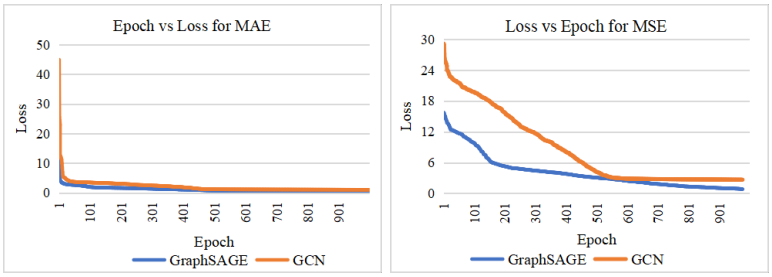
2. Different traffic states (e.g., traffic jams)
3. Typical positioning error ¡ 10 cm

The dataset includes data extracted from 60 recordings. For each recording, the dataset contains an image of the recorded highway section, the recording location, an overview of recorded vehicle tracks and the tracks' trajectories. Each track covered about 420m in a six-lane bidirectional highway.
In this section, the experimental results of the GCN and GraphSAGE models that are trained on the German highD dataset and the performance analysis of each model is presented. As described in section 3.2.2, a graph was constructed for each location given in the dataset. We conducted our experiments on six different locations, each having a distinct number of tracks. For the first location, we considered 1410 tracks and constructed a graph based on the collected data. Similarly, we analyzed 914 tracks for the second location to create the graph. For location 3, we considered 1163 tracks; for location 4, the number of tracks taken into account was 855. For the remaining two locations, 607 and 714 tracks were analyzed, respectively. In total, six graphs were constructed for six different locations. In each graph, the number of nodes corresponds to the number of tracks considered for analysis, and connections between nodes were established based on the presence of four-point intersections. This approach allowed us to effectively model the spatial relationships between different tracks and uncover insights about the behavior of vehicles in each location. The road

network graph used in training and testing was the same for each location. From the dataset, based on the average velocity of the track the peak, off-peak and normal hour was separated for each location. As the actual dataset does not contain the information about this, we considered peak hour when the velocity was the lowest in a time span, similarly off-peak hour was considered when the velocity was the greatest. Though a location is divided into three different dataset the graph for a location was not changed. Then the GCN and the GraphSAGE models were trained on each location's graph. Both models were trained for 2000 epochs at a learning rate 0.01. Then, we tested the models on each location's corresponding graph on a different road condition and time than those included in the training set. The training and testing results obtained for the two models are elaborately described later in this section and comparison between them is presented in section 4.4.
Next, the A* search algorithm was used to find the quickest path. The source and the destination track numbers were provided as input to this shortest path algorithm. The predicted time of the model that produced better results were assigned as weights to the algorithm.

4.2.2 Loss Curves

The loss versus the number of epochs curves for training data at location 2 is presented in Fig. 6, utilizing MAE and MSE for both the GCN and GraphSAGE models. In Fig. 6(a), a comparison between the two models is depicted using MAE, while MSE is employed in Fig. 6(b). Notably, for MAE, both the GCN and GraphSAGE models initially faced a rapid reduction in loss from high to low after a few epochs, followed by a flattening of the curves. Observing the curves, it is evident that the loss of the GraphSAGE model consistently remains lower than that of the GCN model. After flattening, the curves oscillate between minima throughout the remaining epochs.
For MSE, the GraphSAGE model consistently outperforms the GCN model.



(a) Mean Absolute Error loss function. (b) Mean Squared Error loss function.

Figure 6: Loss vs Epoch Curve for Location 2

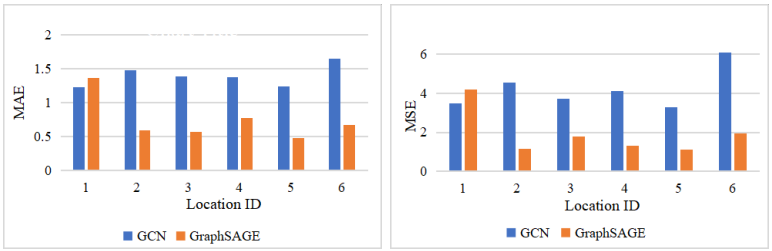
After 500 epochs, the GCN attempts to catch up with the GraphSAGE model but eventually cannot further reduce the loss. Both curves gradually decrease over time, indicating a slow, epoch-by-epoch decrease in loss, unlike MAE, which

does not exhibit a sharp decline. The fluctuations in the loss values may suggest that the model is oscillating in its attempt to converge. Overall, both models show improvements over time, but the use of different loss functions affects the rate and pattern of the loss values. The other locations also exhibit similar characteristics in their loss curves; that is why only location 2 is selected for showcasing.

4.2.3 R-squared And Loss Function (MAE and MSE) Analysis

In the case of the comparison between GraphSAGE and GCN models, the R-squared values for both models were compared in the training data using both MAE and MSE loss functions for all locations. The results in Fig. 7 showed that the R-squared values for GraphSAGE were greater than GCN in both cases, indicating that the GraphSAGE model is better at explaining the variance in the target variable (i.e., predicting the time) in the training data.

Prediction results are achieved from time calculated in the test data and the



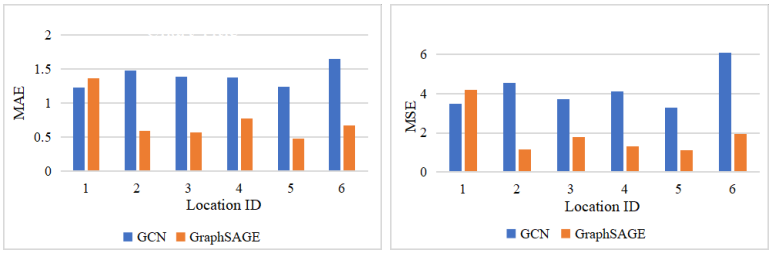
(a) Mean Absolute Error loss function. (b) Mean Squared Error loss function.

Figure 7: Comparison between R-squared values of GCN and GraphSAGE.

time predicted from the models. The bar charts in Fig. 8 show the comparison of Mean Absolute Error (MAE) and Mean Squared Error (MSE) for the GCN and GraphSAGE models for all six locations. From the bar charts, it can be concluded that the GraphSAGE model outperforms the GCN model in terms of both MAE and MSE. The MAE and MSE values for the GraphSAGE model are consistently lower than those for the GCN model. This suggests that the GraphSAGE model is better at predicting the time than the GCN model for this particular dataset and experimental setup. Therefore, we would choose the GraphSAGE model for the task.

4.3 Performance Analysis on a Road Network of Khulna City, Bangladesh

Khulna, situated in the southwestern part of Bangladesh, is the third-largest city in the country. This case study will evaluate the city's basic road system and identify potential areas for development. Khulna has a population of about 1.5 million, is a metropolis expanding rapidly. The city is an essential economic



(a) Mean Absolute Error loss function. (b) Mean Squared Error loss function.

Figure 8: Prediction error evaluated on test datasets for location 1 to 6.

hub in Bangladesh, with a strong industrial base and a thriving port. The road network in the city consists of several major roads that connect various parts of the city, as well as smaller local roads. In the case study, the roads and vehicle information of the city will be taken and for a particular case the start and the target location will be given. The route that the vehicle should take will be shown.

4.3.1 Road Network Analysis and Dataset Generation

The proposed methodology in this study was tested on real-world data collected for a portion of Khulna, Bangladesh. The road network data was obtained from OpenStreetMap and a subset of the road segments was manually marked with integers from 1 to 502 to construct a graph with 502 nodes. The edge list of the graph was created based on the real data and the relationships among each road segment were taken into consideration. This approach allowed for the use of actual road data for the analysis, making the results more applicable to real-world scenarios. By utilizing real-world data, the methodology was able to accurately predict the time needed for passing each road segment and find the quickest route for emergency vehicles. Overall, the use of local data from Khulna, Bangladesh provides a relevant and practical example of how machine learning and graph theory can be used to solve transportation-related problems in a specific region.

The road network in Khulna is relatively simple, with a few major roads running through the city, such as Rupsa-Khulna Road, Jessore Road, and KDA Avenue. The smaller local roads connect the residential areas to the main roads, but many roads are in poor condition, especially in residential areas, which can be dangerous and contribute to traffic congestion. Additionally, there are limited facilities for non-motorized road users.

Fig. 9 depicts a map of a part of Khulna city from New Market to Shonadanga, showing the hospital location and a yellow circle indicating the house of someone who wants to reach the hospital. Several possible ways of reaching the hospital are shown, and using route optimization, the path that will take the least amount of time can be selected. The shortest distance may not necessarily be the fastest route due to road conditions, so time predictions were made to

Figure 9: Map of a road network of Khulna city.

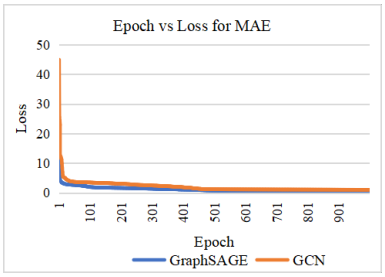
determine the best route to take.

4.3.2 Graph Construction and Results Analysis

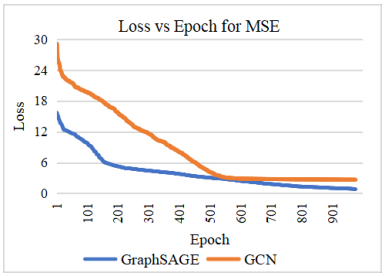
Here are the steps for constructing the graph of a road network in Khulna city:

- Selection of area: We selected a specific area of Khulna city for our experiment.
- Map export: We exported the map of the selected area from OpenStreetMap, which provided us with a network of roads.
- Road segment numbering: Each road segment in the network was marked with a unique integer number for identification purposes.
- Determination of connections: We determined the connections between each road segment and any other road segments, covering all the intersections.
- Node placement: Each road segment was placed as a node in the graph.
- Edge Creation: All connections between the road segments were established as edges in the graph.

Following these steps, we constructed the graph for the selected area, which was then used for optimal route generation. The loss curves for both GCN and GraphSAGE models for Khulna city are depicted in Fig 10. They appear to have similar trends as their counterparts for other locations. Specifically, the loss values for GraphSAGE models start at higher values than those for GCN but eventually converge to lower values than GCN for MAE and for MSE the GraphSAGE always performs better. Overall, the loss values for GraphSAGE models are lower than GCN for both loss functions.



(a) Mean Absolute Error loss function.



(b) Mean Squared Error loss function.

Figure 10: Loss vs Epoch curve for Khulna city Dataset

4.4 Performance Comparison

4.4.1 Prediction accuracy comparison between GCN and GraphSAGE models

In table 4, the performance comparison of GCN and GraphSAGE models is stated based on three evaluation metrics: R-squared, Mean Absolute Error (MAE), and Mean Squared Error (MSE). The R-squared value is a measure of the goodness of fit of a model, with a higher value indicating a better fit. The MAE and MSE are measures of the difference between the predicted values and the actual values. A lower value of MAE and MSE indicates that the model is making more accurate predictions. In table 4, location id of 1 to 6 indicate the six different locations of the benchmark dataset used and location id 7 is used to indicate Khulna.

Table 4: **Prediction accuracy comparison between GCN and GraphSAGE models.**

	GCN								GraphSAGE							
	Evaluation on Train Set		Evaluation on Test Set		Off-Peak		Peak		Evaluation on		Evaluation on		Off-Peak		Peak	
Location ID	R-squared (in MAE)	R-squared (in MSE)	MAE	MSE	MAE	MSE	MAE	MSE	R-Squared (in MAE)	R-Squared (in MSE)	MAE	MSE	MAE	MSE	MAE	MSE
1	0.6707	0.6433	1.2571	3.4623	2.152	5.572	8.696	8.73	0.7185	0.7204	1.3653	4.2141	1.5846	3.1526	7.397	7.6827
2	0.6941	0.6889	1.4810	4.5450	0.864	1.61	1.378	3.345	0.9001	0.8409	0.5970	1.1416	0.5932	0.67	0.8387	1.2579
3	0.6392	0.6426	1.3848	3.7224	1.035	2.177	1.366	3.271	.9236	0.8394	0.5701	1.8021	0.6081	0.6596	0.9195	1.6156
4	0.6966	0.7028	1.3824	4.1117	0.976	1.572	1.503	4.179	0.8889	0.8255	0.7709	1.3148	0.7843	1.1514	0.9571	1.769
5	0.7397	0.7525	1.2475	3.2993	1.145	2.446	1.246	3.076	0.8934	0.7895	0.4842	1.1246	1.2472	2.5764	1.1276	2.1756
6	0.5385	0.5736	1.6478	6.0797	1.060	2.535	1.712	6.997	0.8192	0.7768	0.6740	1.9468	0.9145	1.6914	1.57	5.4593
7 (khulna)	0.4687	0.5271	2.0403	6.4333	1.4233	3.8071	1.9560	5.8174	0.9317	0.8600	0.5065	1.4295	0.8810	1.3939	1.0468	2.0483

Based on the results, the GraphSAGE model outperformed the GCN model significantly. The GraphSAGE model yielded higher R-squared values for multiple locations, signifying its suitability for the data. Moreover, it demonstrated lower MAE and MSE values compared to the GCN model, indicating more precise predictions. Consequently, the results suggest that the GraphSAGE model is a superior fit for the data and a reliable choice for future predictions. In the subsequent step, only predictions from the GraphSAGE model were utilized. The model predicted the time required to cross each track, where the generated

times were assigned as the weights for each edge in the graph network. Subsequently, an optimal route was generated based on these inputs by employing the A* shortest path algorithm.

4.4.2 Optimal Route Generation

The study evaluated the performance of three routing algorithms across two distinct datasets: the HighD dataset and the road network of Khulna city. The algorithms under scrutiny were the A* search algorithm alone, A* enhanced with Graph Convolutional Network (GCN), and A* augmented with GraphSAGE. These were tested under three traffic conditions: off-peak, normal, and peak. Importantly, the study also determined the best route for Emergency Vehicles (EVs) according to a priority score.

Table 5: **Prediction accuracy comparison between GCN and GraphSAGE models.**

				Only A*		**GCN + A***			**GraphSAGE + A***			
Location ID	Source	Destination	Road Traffic Type	Optimal Route(Shortest distance)	Optimal Time(s)	Optimal Route (shortest time)	Optimal Time(s)	Time savings (compared to A* only)	Optimal Route(shortest time)	Optimal Time (s)	Time savings (compared to A* only)	Best Route for EVs according to priority score
1	11	25	Off-peak	11,12,15, 16, 17, 20, 21, 22, 25	83	11, 12, 15, 16, 17, 20, 21, 22, 25	83	-	11, 12, 15, 16, 17, 20, 21, 22, 25	83	-	-
			Normal		156.86	11, 14, 15, 16, 19, 20, 21, 24, 25	115.279	41.581s	11,12,15,16,18,20,21 ,23,25	113.39s	43.47s	Only A*
			Peak		229.017	11, 12, 15, 16, 17, 20, 21, 22, 25	215.514	13.503s	11, 12, 15, 16, 17, 20, 21, 24, 25	211.387s	17.63s	GraphSAGE +A*
2	62	85	Off-peak	62, 65, 66, 67, 70, 71, 72, 75, 76, 77, 80, 81, 82, 85	172.85	62, 65, 66, 67, 70, 71, 72, 75, 76, 77, 80, 81, 82, 85	172.85	-	62, 65, 66, 67, 70, 62, 65, 66, 67, 70, 80, 81, 82, 85	172.85	-	-
			Normal		233.1	62, 65, 66, 67, 70, 71, 73, 75, 76, 78, 80, 81, 82, 85	189.797	43.303s	62, 65, 66, 67, 70, 71, 74, 75, 76, 78, 80, 81, 84, 85	183.84	49.26s	GraphSAGE + A*
			Peak		248.8	62, 65, 66, 69, 70, 71, 72, 75, 76, 78, 80, 81, 82, 85	215.592	33.208s	62, 65, 66, 67, 70, 71, 73, 75, 76, 78, 80, 81, 83, 85	216.734	32.066s	GraphSAGE + A*
3	520	550	Off-Peak	520.,521, 522,525,	207.1	520, 521, 522, 525,526, 527, 530, 531, 532, 535, 536, 537, 540, 541, 542, 545, 546, 547, 550	207.1	-	520, 521, 522, 525, 526, 527, 530, 531, 532, 535, 536, 537, 540, 541, 542, 545, 546, 547, 550	207.1	-	-
			Normal	526, 527, 530, 531, 532, 535,	237.8	520, 521, 522, 525, 526, 527, 530, 531, 532, 535, 536, 537, 540, 541, 542, 545, 546, 547, 550	232.47	5.33s	520, 521, 523, 525, 526, 529, 530, 531, 532, 535, 536, 537, 540, 541, 542, 545, 546, 547, 550	212.12	25.68s	GraphSAGE + A*
			Peak	542, 545, 546, 547, 550 536, 537, 540, 541,	256.6	520, 521, 524, 525, 526, 527, 530, 531, 534, 535, 536, 539, 540, 541, 542, 545, 546, 549, 550	251.416	5.18s	520, 521, 522, 535, 526, 527, 530, 531, 532, 535, 536, 539, 540, 541, 542, 545, 546, 547, 550	217.59	39.01s	Only A*
4	49	83	Off-peak	52, 55, 56, 57, 60, 61, 62, 65, 66, 67, 70, 71, 72, 75, 76, 77, 80, 81, 83	223.14	49, 50, 51, 52, 55, 56, 57, 60, 61, 62, 65, 66, 67, 70, 71, 72, 75, 76, 77, 80, 81, 83	223.14	-	49, 50, 51, 52, 55, 56, 57, 60, 61, 62, 65, 66, 67, 70, 71,72, 75, 76, 77, 80,81,83	223.14	-	-
			Normal		394.18	49, 50, 51, 54, 55, 56, 59, 60, 61, 62, 65, 66, 67, 70, 71, 74, 75, 76, 79, 80, 81, 83	268.267	125.913s	49, 50, 51, 52, 55, 56, 57, 60, 61, 62, 65, 66, 67, 70, 71,72, 75, 76, 77, 80,81,83	242.03	152.15s	GraphSAGE +A*
			Peak		416.25	49, 50, 51, 54, 55, 56, 59, 60, 61, 62, 65, 66, 67, 70, 71, 74, 75, 76, 79, 80, 81, 83	309.602	106.648s	49, 50, 51, 52, 55, 56, 57, 60, 61, 64, 65, 66, 68, 70, 71,74, 75, 76, 79, 80,81,83	299.028	117.22s	Only A*
5	10	47	Off-Peak	10, 11, 12,15,16, 17,20,21,22,	236.23	10, 11, 12, 15, 16, 17, 20, 21, 22, 25, 26, 27, 30, 31, 32, 35, 36, 37, 40, 41, 42, 45, 46, 47	236.23	-	10, 11, 12, 15, 16, 17, 20, 21, 22, 25,26,27 30, 31, 32,35,36, 37, 40, 41, 42 45,46,47	236.23	-	-
			Normal	25,26,27,30, 31,32,35,36, 37,40,41, 42,45, 46, 47	299.679	10, 11, 14, 15, 16, 17, 20, 21, 22, 25,26,29, 30, 31, 32, 35, 36, 39, 40, 41, 42, 45, 46, 47	282.47	17.203s	10, 11, 13, 15, 16, 18, 20, 21, 23, 25,26,27, 30, 31, 34, 35,36,39,40, 41, 42, 45, 46, 47	271.51	28.168s	GraphSAGE + A*
			Peak		383.03	10, 11, 12, 15, 16, 17, 20, 21, 22, 25, 26, 27, 30, 31, 34, 35, 36, 37, 40, 41, 44, 45, 46, 47	381.62	1.41s	10, 11, 12, 15, 16,18, 20, 21,24, 25,26, 28, 30, 31, 34, 35, 36, 37, 40, 41, 42, 45, 46, 47	371.671	11.359s	GraphSAGE + A*
6	118	140	Off-Peak	118,120,121, 122, 125, 126,	151.249	118, 120, 121, 122, 125, 126, 127, 130, 131, 132, 135, 136, 137, 140	151.249	-	118, 120, 121, 122, 125, 126, 127, 130, 131, 132, 135, 136, 137, 140	151.249	-	-
			Normal	127,130,131, 132, 135, 136, 137, 140	151.249	118, 120, 121, 122, 125, 126, 129, 130, 131, 132, 135, 136, 139, 140	151.249	-	118, 120, 121, 122, 125, 126, 127, 130, 131, 132, 135, 136, 137, 140	166.96	3.35s	Only A*
			Peak		291.97	118, 120, 121, 122, 125, 126, 129, 130, 131, 132, 135, 136, 137, 140	219.64	72.33s	118, 120, 121, 122, 125, 126, 129, 130, 131, 132, 135, 136, 137, 140	195.510	96.46s	Only A*

In the HighD dataset, a clear pattern emerged across the six route scenarios examined. During off-peak hours, all three algorithms consistently produced

identical results, suggesting that the sophistication of graph neural network enhancements offers little advantage when traffic flow is minimal. However, as traffic density increased, the neural network-enhanced algorithms began to demonstrate their efficacy. Under normal traffic conditions, both GCN+A* and GraphSAGE+A* consistently outperformed the standalone A* algorithm. The time savings achieved ranged from a modest 5.33 seconds to a substantial 152.15 seconds. Notably, GraphSAGE+A* emerged as the top performer in almost all of the cases, while GCN+A* led in one instance with very little difference. This trend continued into peak traffic scenarios, where time savings extended from 1.41 seconds to 117.22 seconds. While both GCN+A* and GraphSAGE+A* generally outperformed A* alone, the best route for EVs varied. For normal traffic conditons, in 2 out of 6 cases, the A* alone route was deemed best for Evs and for the rest cases GraphSAGE+A* was choosen as best path for EVs. For peak conditions, The results were mixed. GraphSAGE+A* was best for EVs in 3 out of 6 cases and only A* was best for the rest. In table 5, overall perfomances with the optimal route is depicted.

Table 6: **Some examples of optimized routes that were generated from the A* search algorithm and using predicted time of GCN and GraphSAGE (for the road network of Khulna city).**

Source	Destination	Road Traffic Type	Only A*		GCN + A*			GraphSAGE + A*			Best Route for EVs according to priority score
			Optimal Route(Shortest distance)	Optimal Time(s)	Optimal Route (shortest time)	Optimal Time(s)	Time savings (compared to A* only)	Optimal Route(shortest time)	Optimal Time (s)	Time savings (compared to A* only)	
85	200	Off-peak	103, 106, 198, 21, 22, 25 197, 212, 216 194, 189, 190, 191, 202, 200	222.89	85, 86, 89, 93, 95, 97,101,103, 106, 198, 197, 212,216, 194, 189, 190, 191, 202,200	222.89	-	85, 86, 89, 93, 95, 97,101, 103, 106, 198, 197,212, 216 212, 216, 194, 189, 190,191, 202, 200	222.89	-	-
		Normal		352.941	85, 87, 52, 53, 41, 27, 206, 209, 211, 198, 197, 212, 216, 194, 189, 190, 191, 202, 200	246.678	106.263s	85, 86, 90, 117, 116, 111, 110, 108, 106, 198, 197, 212, 216, 194, 189, 190, 191, 202, 200	237.474	115.467s	Only A*
		Peak		428.57	85, 87, 52, 40, 28, 27, 206, 209, 211, 198, 197, 212, 216, 194, 189, 190, 191, 202, 200	428.57	164.953s	85, 87, 52, 40, 28, 27, 206, 209, 211, 198, 197, 212, 216, 194, 189, 190, 191, 202, 200	262.367s	166.203s	Only A*
20	127	Off-peak	20, 21, 6, 27, 41, 42, 57, 58, 80, 82, 125, 127	222.89	20, 21, 6, 27, 41, 42, 57, 58, 80, 82, 125, 127	222.89	-	20, 21, 6, 27, 41, 42, 57, 58,80, 82, 125, 127	222.89	-	-
		Normal		142.64	20, 24, 6, 8, 11, 112, 111, 116, 119, 121, 123, 124, 127	246.678	106.263s	20, 21, 6, 8, 11 112, 111, 116, 119, 121, 123, 124, 127	126.844	15.796s	GraphSAGE +A*
		Peak		190.47	20, 21, 6, 8, 11, 112, 111, 116, 119, 121, 123, 124, 127	161.398	29.072s	820, 22, 6, 8, 11, 112, 111, 116, 119, 121, 123, 124, 127	172.746	17.724s	GCN + A*
67	389	Off-peak	67, 66, 68, 74, 46, 56, 42, 41, 43, 44, 50, 32, 33, 395, 394, 389	182.35	67, 66, 68, 74, 46, 56, 42, 41, 43, 44, 50, 32, 33, 395, 394, 389	182.35	-	67, 66, 68, 74, 46, 56, 42, 41, 43, 44, 50, 32, 33, 395, 394, 389	182.35	-	-
		Normal		294.91	66, 68, 60, 59, 57, 42, 41, 43, 44, 50, 32, 33, 395, 394, 389	202.433	92.477s	67, 66, 68, 74, 46, 56, 42, 41, 43, 44, 50, 32, 33, 395, 394, 389	197.765	97.145s	GraphSAGE +A*
		Peak		357.14	67, 66, 68, 60, 59, 57, 42, 41, 43, 44, 50, 32, 33, 395, 394, 389	235.031	122.109s	67, 66, 68, 74, 46, 56, 42, 41, 43, 44, 50, 32, 33, 395, 394, 389	222.229	134.911s	GraphSAGE +A*

The Khulna city road network dataset yielded similar patterns, albeit with some nuances. Off-peak results mirrored those of the HighD dataset, with all algorithms producing identical outcomes. In normal traffic conditions, the graph-based models once again demonstrated superiority, with time savings spanning from 4.41 to 115.467 seconds. GraphSAGE+A* exhibited the best performance in time savings. Peak traffic scenarios in the Khulna dataset showed a more balanced distribution of optimal performances. GCN+A* and GraphSAGE+A* maintained their edge over the standalone A* algorithm, achieving time savings between 17.724 and 166.203 seconds. For the best route of EVs in normal conditions, GraphSAGE+A* was considered the best route for EVs in 2 out of 3 cases. A* alone was best for EVs in 1 case. However, in peak conditions, GCN+A* was selected as the best path selector for one case. Table 6 illustrated all the informations of testing results in Khulna Datase. The key observations

from the findings include the following: The optimal routes for emergency vehicles (EVs) often differed from those that offered the shortest travel times, especially during both normal and peak traffic conditions. This highlights the significance of the priority score, which takes into account safety along with time efficiency. Although the A* algorithm typically resulted in longer travel times, it was frequently regarded as the best choice for EVs, a conclusion that largely depended on the specific dataset used. This preference for A* indicates that routing decisions for EVs consider factors beyond just travel time. While neural network-enhanced algorithms occasionally provided quicker routes, they did not consistently align with the priority score that favors EVs. Interestingly, during off-peak conditions, there was no notable distinction between the algorithms, nor was there a clear preference for any particular EV route.

5 Conclusion

The study introduces a novel method for optimizing routes for emergency vehicles by integrating spatial and temporal traffic data with advanced graph-based algorithms. Utilizing Graph Convolutional Networks (GCN) and GraphSAGE models for time prediction, the study pairs these with the A* search algorithm to select the most efficient routes. A key contribution is the development of a prioritization system for emergency vehicles, which evaluates routes based on traffic flow, safety metrics, and overall suitability for urgent scenarios. The results demonstrate that this hybrid approach not only reduces travel time for any vehicle but also ensures that emergency vehicles are guided through routes that maximize speed and safety, enhancing the effectiveness of emergency response. There are also limitations to the study that need to be addressed in future research. Future research directions may include additional factors such as road infrastructure, weather conditions, and real-time traffic updates.The use of advanced machine learning algorithms, such as reinforcement learning or deep learning, could also be explored to improve the performance of the proposed approach. Overall, there is significant potential for future research in the optimal route generation for emergency vehicles, and this study serves as a foundation for further advancements in the area.

References

[1] José Nelas and Joana Dias. Locating emergency vehicles: modelling the substitutability of resources and the impact of delays in the arrival of assistance. *Operations Research Perspectives*, 8:100202, 2021.

[2] Shridevi Jeevan Kamble and Manjunath R Kounte. A survey on emergency vehicle preemption methods based on routing and scheduling. *International Journal of Computer Networks and Applications*, 9(1):60–71, 2022.

[3] Isradi Zainal, Fatma Lestari, Satriadi Gunawan, Andrio Adiwibowo, Abdul Kadir, and Noor Aulia Ramadhan. Fire vehicle route, response time, and service coverage optimizations in pekojan urban village, tambora subdistrict fire hotspot of jakarta city indonesia. *PREPOTIF: Jurnal Kesehatan Masyarakat*, 6(2):1454–1468, 2022.

[4] Jiandong Zhao, Yujie Guo, and Xiaohong Duan. Dynamic path planning of emergency vehicles based on travel time prediction. *Journal of advanced transportation*, 2017(1):9184891, 2017.

[5] Yinli Jin, Wanrong Xu, Ping Wang, and Jun Wang. Route planning of rescue vehicles in the process of dynamic change of traffic volume under emergency conditions. In *2019 3rd International Symposium on Autonomous Systems (ISAS)*, pages 125–130. IEEE, 2019.

[6] Jiaming Wu, Balázs Kulcsár, Soyoung Ahn, and Xiaobo Qu. Emergency vehicle lane pre-clearing: From microscopic cooperation to routing decision making. *Transportation research part B: methodological*, 141:223–239, 2020.

[7] Marcus Poulton, Anastasios Noulas, David Weston, and George Roussos. Modeling metropolitan-area ambulance mobility under blue light conditions. *IEEE Access*, 7:1390–1403, 2018.

[8] Ran Tian, Shanwei Li, and Guoying Yang. Retracted article: Research on emergency vehicle routing planning based on short-term traffic flow prediction. *Wireless Personal Communications*, 102(2):1993–2010, 2018.

[9] Ziling Zeng, Wen Yi, Shuaian Wang, and Xiaobo Qu. Emergency vehicle routing in urban road networks with multistakeholder cooperation. *Journal of transportation engineering, Part A: Systems*, 147(10):04021064, 2021.

[10] Yipeng Liu, Haifeng Zheng, Xinxin Feng, and Zhonghui Chen. Short-term traffic flow prediction with conv-lstm. In *2017 9th international conference on wireless communications and signal processing (WCSP)*, pages 1–6. IEEE, 2017.

[11] Mehdi Hosseinzadeh, Bruno Sinopoli, Ilya Kolmanovsky, and Sanjoy Baruah. Mpc-based emergency vehicle-centered multi-intersection traffic control. *IEEE Transactions on Control Systems Technology*, 31(1):166–178, 2022.

[12] Nikki Rathore, Pramod Kumar Jain, and Manoranjan Parida. A routing model for emergency vehicles using the real time traffic data. In *2018 IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI)*, pages 175–179. IEEE, 2018.

[13] Xiangyang Ren, Shuai Chen, and Liyuan Ren. Optimization of regional emergency supplies distribution vehicle route with dynamic real-time demand. *Math. Biosci. Eng*, 20:7487–7518, 2023.

[14] Nasser A El-Sherbeny. The algorithm of the time-dependent shortest path problem with time windows. *Applied Mathematics*, 5(17):2764–2770, 2014.

[15] Ufuk Kula, Ozden Tozanli, and Saniye Tarakcio. Emergency vehicle routing in disaster response operations. In *POMS 23rd Annual Conference, Chicago (April 20-23)*, 2012.

[16] Yina Yuan, Xiaoguang Zhou, and Mengke Yang. Emergency logistics vehicle routing optimization based on insufficient supply. In *E3S Web of Conferences*, volume 136, page 04068. EDP Sciences, 2019.

[17] Jin Qin, Yong Ye, Bi-rong Cheng, Xiaobo Zhao, and Linling Ni. The emergency vehicle routing problem with uncertain demand under sustainability environments. *Sustainability*, 9(2):288, 2017.

[18] Rodrigo Moretti Branchini, Vinícius Amaral Armentano, and Arne Løkketangen. Adaptive granular local search heuristic for a dynamic vehicle routing problem. *Computers & Operations Research*, 36(11):2955–2968, 2009.

[19] Qiang Zhang and Shengwu Xiong. Routing optimization of emergency grain distribution vehicles using the immune ant colony optimization algorithm. *Applied Soft Computing*, 71:917–925, 2018.

[20] Zahra Gharib, Ali Bozorgi-Amiri, Reza Tavakkoli-Moghaddam, and Esmaeil Najafi. A cluster-based emergency vehicle routing problem in disaster with reliability. *Scientia iranica*, 25(4):2312–2330, 2018.

[21] Ruisong Liu and Ning Wang. Data-driven bus route optimization algorithm under sudden interruption of public transport. *IEEE Access*, 10:5250–5263, 2022.

[22] Jiujun Cheng, Huaichen Yan, Aiguo Zhou, Chunmei Liu, Ding Cheng, Shangce Gao, Di Zang, and Deli Cheng. Location prediction model based on the internet of vehicles for assistance to medical vehicles. *IEEE Access*, 8:10754–10767, 2019.

[23] Noraimi Azlin Mohd Nordin, Zati Aqmar Zaharudin, Mohd Azdi Maasar, and Nor Amalina Nordin. Finding shortest path of the ambulance routing: Interface of a algorithm using c# programming. In *2012 ieee symposium on humanities, science and engineering research*, pages 1569–1573. IEEE, 2012.

[24] Aoyu Liu and Yaying Zhang. Spatial-temporal interactive dynamic graph convolution network for traffic forecasting. *arXiv preprint arXiv:2205.08689*, 2022.

[25] Zongtao Duan, Kai Zhang, Zhe Chen, Zhiyuan Liu, Lei Tang, Yun Yang, and Yuanyuan Ni. Prediction of city-scale dynamic taxi origin-destination flows using a hybrid deep neural network combined with travel time. *IEEE Access*, 7:127816–127832, 2019.

[26] Redouane Benabdallah Benarmas and Kadda Beghdad Bey. A deep learning-based framework for road traffic prediction. *The Journal of Supercomputing*, 80(5):6891–6916, 2024.

[27] Van-Thuan Tran and Wei-Ho Tsai. Acoustic-based emergency vehicle detection using convolutional neural networks. *IEEE Access*, 8:75702–75713, 2020.

[28] Ping Wang, Wenbang Hao, and Yinli Jin. Fine-grained traffic flow prediction of various vehicle types via fusion of multisource data and deep learning approaches. *IEEE Transactions on Intelligent Transportation Systems*, 22(11):6921–6930, 2020.

[29] Jiao-Hong Yi, Jian Wang, and Gai-Ge Wang. Using monarch butterfly optimization to solve the emergency vehicle routing problem with relief materials in sudden disasters. *Open Geosciences*, 11(1):391–413, 2019.

[30] Prakash Rosayyan, SenthilKumar Subramaniam, and Saravana Ilango Ganesan. Decentralized emergency service vehicle pre-emption system using rf communication and gnss-based geo-fencing. *IEEE Transactions on Intelligent Transportation Systems*, 22(12):7726–7735, 2020.

[31] Van-Linh Nguyen, Ren-Hung Hwang, and Po-Ching Lin. Controllable path planning and traffic scheduling for emergency services in the internet of vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 23(8):12399–12413, 2021.

[32] Gaby Joe Hannoun, Pamela Murray-Tuite, Kevin Heaslip, and Thidapat Chantem. Facilitating emergency response vehicles' movement through a road segment in a connected vehicle environment. *IEEE Transactions on Intelligent Transportation Systems*, 20(9):3546–3557, 2018.

[33] Ying Qiu, Meng Shi, Xinna Zhao, and Yongping Jing. System dynamics mechanism of cross-regional collaborative dispatch of emergency supplies based on multi-agent game. *Complex & Intelligent Systems*, pages 1–12, 2021.

[34] Weiqi Yu, Weichen Bai, Wenjing Luan, and Liang Qi. State-of-the-art review on traffic control strategies for emergency vehicles. *IEEE Access*, 10:109729–109742, 2022.

[35] Ryo Katsuma and Shin Yoshida. Dynamic routing for emergency vehicle by collecting real-time road conditions. *International Journal of Communications, Network and System Sciences*, 11(2):27–44, 2018.

[36] Omar Adil Mahdi, Jabbar Abed Eleiwy, Yusor Rafid Bahar Al-Mayouf, and Bourair AL-Attar. An efficient node selection algorithm in the context of iot-based vehicular ad hoc network for emergency service. *Journal of Intelligent Systems*, 33(1):20240208, 2024.

[37] Pengying Ouyang, Pan Liu, Yanyong Guo, and Kequan Chen. Effects of configuration elements and traffic flow conditions on lane-changing rates at the weaving segments. *Transportation research part A: policy and practice*, 171:103652, 2023.

[38] Robert Krajewski, Julian Bock, Laurent Kloeker, and Lutz Eckstein. The highd dataset: A drone dataset of naturalistic vehicle trajectories on german highways for validation of highly automated driving systems. In *2018 21st international conference on intelligent transportation systems (ITSC)*, pages 2118–2125. IEEE, 2018.